Session initialization

Every BF session **must** start by importing `bf` and initializing the BF object. This single object is the gateway to everything: data handling, model definition, fitting, diagnostics, and saving.

> **Rule:** Always import `bf` **before** importing `jax` or any other JAX-dependent module, so that the XLA configuration is applied first.

Python

```python
from BayesForge import bf

m = bf(
    platform="cpu",          # "cpu", "gpu", or "tpu"
    cores=None,              # CPU cores (None = all available)
    float_precision=64,      # 64 for float64, 32 for float32
    rand_seed=42,            # int for reproducibility, True for random
    gpu_index=None,          # GPU index when platform="gpu"
    deallocate=False,        # deallocate existing device first
    print_devices_found=True,# print device info on init
)
```

R

```r
library(BayesForge)

m <- importBF(
    platform = "cpu",            # "cpu", "gpu", or "tpu"
    cores = NULL,                # CPU cores (NULL = all available)
    rand_seed = 42,              # integer for reproducibility, TRUE for random
```

```
    deallocate = FALSE,           # deallocate existing device first
    print_devices_found = TRUE    # print device info on init
)
```

Julia

```
using PyBayesForge

m = importBF(
    platform = "cpu",             # "cpu", "gpu", or "tpu"
    cores = nothing,              # CPU cores (nothing = all available)
    rand_seed = 42,               # integer for reproducibility, true for random
    deallocate = false,           # deallocate existing device first
    print_devices_found = true,   # print device info on init
)
```

Arguments

Argument	Type	Default	Description
`platform`	str	`"cpu"`	Hardware backend: `"cpu"`, `"gpu"`, or `"tpu"`
`cores`	int	`None`	CPU cores to allocate (`None` = all). Only for `platform="cpu"`
`float_precision`	int or str	`64`	JAX floating-point precision. Pass `64` (`"float64"`) or `32` (`"float32"`)
`rand_seed`	int or bool	`True`	Reproducibility seed. Pass an `int` (e.g. `42`) for reproducible results, or `True` for entropy-based randomness
`gpu_index`	int	`None`	Which GPU to use by index. Only for `platform="gpu"`
`deallocate`	bool	`False`	Deallocate existing device before setting up new configuration
`print_devices_found`	bool	`True`	Print detected devices on initialization

Argument	Type	Default	Description
`backend`	str	`"numpyro"`	Inference backend: `"numpyro"` or `"tfp"`

In R and Julia, `importBF()` takes the same arguments except `float_precision` and `gpu_index` (Python only); `NULL` (R) and `nothing` (Julia) stand for Python's `None`.

Common initialization patterns

Python

```
# Reproducible CPU session (default for development)
m = bf(platform="cpu", rand_seed=42, float_precision=64)

# Multi-core CPU for parallel chains
m = bf(platform="cpu", cores=8, rand_seed=42)

# 32-bit precision for memory efficiency
m = bf(platform="cpu", rand_seed=42, float_precision=32)
```

```
# GPU with specific device (Linux/WSL2 with an NVIDIA GPU)
m = bf(platform="gpu", gpu_index=0, rand_seed=42)
```

R

```
# Reproducible CPU session (default for development)
m <- importBF(platform = "cpu", rand_seed = 42)

# Multi-core CPU for parallel chains
m <- importBF(platform = "cpu", cores = 8, rand_seed = 42)
```

```
# GPU (Linux/WSL2 with an NVIDIA GPU; environment created with setup_env(backend = "gpu"))
m <- importBF(platform = "gpu", rand_seed = 42)
```

Julia

```
# Reproducible CPU session (default for development)
m = importBF(platform = "cpu", rand_seed = 42)

# Multi-core CPU for parallel chains
m = importBF(platform = "cpu", cores = 8, rand_seed = 42)
```

```
# GPU (Linux/WSL2 with an NVIDIA GPU; GPU build of JAX added with CondaPkg)
m = importBF(platform = "gpu", rand_seed = 42)
```

Saving and reusing a BF object

After fitting a model, save the complete BF object to disk. This persists the model, data, posteriors, and full sampler state — allowing you to reload and analyze later without re-fitting.

Save

Python

```
m.save("/path/to/my_model.pkl")
# Default filename: "{model_name}_bf.pkl" in current directory
```

R

```
# A model written in R cannot be pickled: m$save() raises
# "TypeError: cannot pickle 'PyCapsule' object". Save the draws instead:
saveRDS(bf_draws(m), "/path/to/my_model_draws.rds")
```

Julia

```
m.save("/path/to/my_model.pkl")
# Default filename: "{model_name}_bf.pkl" in current directory
```

Load

Python

```
m = bf.load("/path/to/my_model.pkl")
```

R

```
draws <- readRDS("/path/to/my_model_draws.rds")   # draws saved from R
# A bf object saved from Python or Julia loads with:
m <- reticulate::import("BayesForge")$bf$load("/path/to/my_model.pkl")
```

Julia

```
using PythonCall
m = PyBayesForge.BFObject(pyimport("BayesForge").bf.load("/path/to/my_model.pkl"))
```

Once loaded, you can access:

Python

```
m.posteriors        # Posterior samples
m.posteriors_full   # Full posterior (before any filtering)
m.summary()         # Posterior summary table
m.data_on_model     # Data used in the fit
m.model             # Model function
m.diag              # Diagnostic tools
```

R

```
m$posteriors        # Posterior samples
m$posteriors_full   # Full posterior (before any filtering)
m$summary()         # Posterior summary table
m$data_on_model     # Data used in the fit
m$model             # Model function
m$diag              # Diagnostic tools
```

Julia

```
m.posteriors        # Posterior samples
m.posteriors_full   # Full posterior (before any filtering)
m.summary()         # Posterior summary table
m.data_on_model     # Data used in the fit
m.model             # Model function
m.diag              # Diagnostic tools
```

Workflow with save/load

Python

```
# --- Session 1: Fit and save ---
import jax.numpy as jnp
from BayesForge import bf
m = bf(platform="cpu", rand_seed=42)

x = jnp.linspace(-1, 1, 40)
y = 0.5 + 2.0 * x + 0.1 * jnp.cos(7 * x)

def model(x, y):
    alpha = m.dist.normal(0, 10, name="alpha")
    beta  = m.dist.normal(0, 1, name="beta")
    sigma = m.dist.exponential(1, name="sigma")
    m.dist.normal(alpha + beta * x, sigma, obs=y)

m.fit(model, obs=dict(x=x, y=y))
m.save("linear_regression.pkl")

# --- Session 2: Load and analyze (no re-fit) ---
from BayesForge import bf
m = bf.load("linear_regression.pkl")
print(m.summary())
print(m.posteriors["beta"].mean())
```

R

```r
# --- Session 1: Fit and save the draws ---
library(BayesForge)
m <- importBF(platform = "cpu", rand_seed = 42)

x <- seq(-1, 1, length.out = 40)
y <- 0.5 + 2.0 * x + 0.1 * cos(7 * x)

model <- function(x, y) {
    alpha <- bf.dist.normal(0, 10, name = "alpha")
    beta  <- bf.dist.normal(0, 1, name = "beta")
    sigma <- bf.dist.exponential(1, name = "sigma")
    bf.dist.normal(alpha + beta * x, sigma, obs = y)
}

m$fit(model, obs = list(x = jnp$array(x), y = jnp$array(y)))
saveRDS(bf_draws(m), "linear_regression_draws.rds")

# --- Session 2: Load and analyze (no re-fit) ---
draws <- readRDS("linear_regression_draws.rds")
print(posterior::summarise_draws(posterior::as_draws_df(draws)))
print(mean(draws$beta))
```

Julia

```julia
# --- Session 1: Fit and save ---
using PyBayesForge, PythonCall
m = importBF(platform = "cpu", rand_seed = 42)

x = collect(range(-1, 1; length = 40))
y = 0.5 .+ 2.0 .* x .+ 0.1 .* cos.(7 .* x)

@BF function model(x, y)
    alpha = m.dist.normal(0, 10, name = "alpha")
    beta  = m.dist.normal(0, 1, name = "beta")
    sigma = m.dist.exponential(1, name = "sigma")
    m.dist.normal(alpha + beta * x, sigma, obs = y)
end

m.fit(model, obs = pydict(x = jnp.array(x), y = jnp.array(y)))
m.save("linear_regression.pkl")
```

```julia
# --- Session 2: Load and analyze (no re-fit) ---
using PyBayesForge, PythonCall
m = PyBayesForge.BFObject(pyimport("BayesForge").bf.load("linear_regression.pkl"))
println(m.summary())
println(m.posteriors["beta"].mean())
```

> **Note:** The object is serialized with `cloudpickle` (from Python and Julia; a model written in R cannot be serialized, so save its draws with `saveRDS(bf_draws(m), ...)`), so the full model closure and sampler state are preserved. The saved `.pkl` file is self-contained and portable.